Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 26.12.2025.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaE_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**
9. **Svaki zadatak nosi po 8 bodova. Kod za testiranje algoritama u Main metodi donosi još jedan bod.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Heap Sort*

Napisati ***Heap Sort*** algoritam za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- Za implementaciju algoritma koristiti klasu ***Heap*** koja se nalazi u template-u
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- Kreirati klasu ***HeapSort*** i u okviru nje generisati metodu ***public static void sort(long[] arr)***

Zadatak 2 – *Kumulativna suma vrednosti levih suseda*

Napisati algoritam koji modifikuje listu tako što vrednost čvora zanemenjuje vrednošću sume svih elemenata koji se u listi nalaze sa njegove leve strane.

Dodatne napomene za rešavanje zadatka:

- Zahtevan potpis metode: ***public void calculateCumulativeSum()***
- Metodu pisati u okviru klase ***LinkedList***

Primer:

- Input: ***LinkedList: 1 -> 0 -> 6 -> 8 -> 4***
- Output: ***0 -> 1 -> 1 -> 7 -> 15***

- Input: ***LinkedList: 1 -> 2 -> 4 -> 0 -> 6***
- Output: ***0 -> 1 -> 3 -> 7 -> 7***

Zadatak 3 – *Hot Potato Game*

Problem: U klasičnoj igri "Vrući krompir", igrači u krugu prebacuju krompir dok muzika svira. Kada muzika stane, igrač koji drži krompir ispada. U našoj **naprednoj varijanti**, broj prolazaka krompira se dinamički menja tokom igre.

Pravila igre:

1. Ima n igrača numerisanih od 1 do n
2. Početni broj prolazaka je K
3. Nakon svakog ispadanja igrača, broj prolazaka za sledeći krug se menja po formuli:

$$novi_prolasci = (stari_prolasci \times 2) \% (preostali_igrači + 1)$$

4. Ako je rezultat formule 0, tada je novi_prolasci = preostali_igrači
5. Igrači se eliminišu dok ne ostane samo jedan - pobednik

<u>Dodatni zahtevi za rešavanje zadatka:</u>

1. Zadatak rešavati u okviru metode: ***public int PlayHotPotato(int totalPlayers, int initialPasses)***
2. Morate koristiti Queue za predstavljanje kruga igrača.
3. Implementirati dinamičko računanje prolazaka po datoj formuli.
4. Rezultat izvršavanja algoritma treba da bude redni broj pobednika u igri.

<u>Primer:</u>

Input 1:	totalPlayers = 7,	initialPasses = 3
Output 1:	expectedWinner = 4	
Input 2:	totalPlayers = 10,	initialPasses = 1
Output 2:	expectedWinner = 8	